

Hands-On Exercises: Authentication and Authorization in ASP.NET Core Web API Microservices

Experiment Title

Authentication and Authorization in ASP.NET Core Web API using JWT (JSON Web Token)

Aim

To implement JWT (JSON Web Token) based Authentication and Authorization in ASP.NET Core Web API Microservices and secure API endpoints using role-based authorization.

Software Requirements

- Visual Studio 2022
- ASP.NET Core Web API (.NET 10)
- Microsoft.AspNetCore.Authentication.JwtBearer
- Windows 10/11

Question 1: Implement JWT Authentication in ASP.NET Core Web API

Scenario

A secure Web API requires users to authenticate before accessing protected resources. JWT authentication is implemented to generate a secure token after successful login.

Steps Performed

1. Created a new ASP.NET Core Web API project.
2. Installed the JWT Bearer NuGet package.
3. Configured JWT settings in appsettings.json.
4. Configured JWT Authentication in Program.cs.
5. Created LoginModel.
6. Created AuthController.
7. Generated JWT Token after successful login.

appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
}
```

```
"AllowedHosts": "*",
```

```
"Jwt": {  
  "Key": "ThisIsASecretKeyForJwtToken123456789",  
  "Issuer": "MyAuthServer",  
  "Audience": "MyApiUsers",  
  "DurationInMinutes": 60  
}  
}
```

Program.cs

```
using Microsoft.AspNetCore.Authentication.JwtBearer;  
using Microsoft.IdentityModel.Tokens;  
using System.Text;  
  
var builder = WebApplication.CreateBuilder(args);  
  
builder.Services.AddControllers();  
builder.Services.AddOpenApi();  
  
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)  
  .AddJwtBearer(options =>  
  {  
    options.TokenValidationParameters = new TokenValidationParameters  
    {  
      ValidateIssuer = true,  
      ValidateAudience = true,  
      ValidateLifetime = true,  
      ValidateIssuerSigningKey = true,  
      ValidIssuer = builder.Configuration["Jwt:Issuer"],  
      ValidAudience = builder.Configuration["Jwt:Audience"],  
      IssuerSigningKey = new SymmetricSecurityKey(  
        Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!))  
    };  
  
    options.Events = new JwtBearerEvents  
    {  
      OnAuthenticationFailed = context =>  
      {  
        if (context.Exception is SecurityTokenExpiredException)  
        {  
          context.Response.Headers.Append("Token-Expired", "true");  
        }  
  
        return Task.CompletedTask;  
      }  
    };  
  });
```

```

builder.Services.AddAuthorization();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

app.UseHttpsRedirection();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();

app.Run();

```

LoginModel.cs

```

namespace JwtAuthDemo.Models;

public class LoginModel
{
    public string Username { get; set; } = "";
    public string Password { get; set; } = "";
}

```

AuthController.cs

```

using JwtAuthDemo.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

namespace JwtAuthDemo.Controllers;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly IConfiguration _config;

    public AuthController(IConfiguration config)
    {
        _config = config;
    }
}

```

```

[HttpPost("login")]
public IActionResult Login(LoginModel model)
{
    if (model.Username == "admin" && model.Password == "12345")
    {
        var token = GenerateJwtToken(model.Username);
        return Ok(new { token });
    }

    return Unauthorized("Invalid username or password");
}

private string GenerateJwtToken(string username)
{
    var claims = new[]
    {
        new Claim(ClaimTypes.Name, username),
        new Claim(ClaimTypes.Role, "Admin")
    };

    var key = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(_config["Jwt:Key"]!)
    );

    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer: _config["Jwt:Issuer"],
        audience: _config["Jwt:Audience"],
        claims: claims,
        expires: DateTime.Now.AddMinutes(
            Convert.ToDouble(_config["Jwt:DurationInMinutes"])
        ),
        signingCredentials: creds
    );

    return new JwtSecurityTokenHandler().WriteToken(token);
}
}

```

Question 2: Secure an API Endpoint Using JWT

Scenario

Only authenticated users should be allowed to access protected resources.

Steps Performed

1. Created SecureController.
2. Applied the [Authorize] attribute.
3. Tested the endpoint with a valid JWT token.

SecureController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace JwtAuthDemo.Controllers;

[ApiController]
[Route("api/[controller]")]
public class SecureController : ControllerBase
{
    [HttpGet("data")]
    [Authorize]
    public IActionResult GetSecureData()
    {
        return Ok("This is protected data.");
    }
}
```

Question 3: Add Role-Based Authorization

Scenario

Only users with the **Admin** role are allowed to access the Admin Dashboard.

Steps Performed

1. Added **Admin** role in JWT claims.
2. Created AdminController.
3. Applied [Authorize(Roles = "Admin")].

AdminController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace JwtAuthDemo.Controllers;

[ApiController]
[Route("api/[controller]")]
public class AdminController : ControllerBase
{
    [HttpGet("dashboard")]
    [Authorize(Roles = "Admin")]
}
```

```

public IActionResult GetAdminDashboard()
{
    return Ok("Welcome to the admin dashboard.");
}
}

```

Question 4: Validate JWT Token Expiry and Handle Unauthorized Access

Scenario

Handle expired JWT tokens and unauthorized requests gracefully.

Steps Performed

1. Configured JWT Bearer Events.
2. Implemented OnAuthenticationFailed.
3. Added custom response header Token-Expired.
4. Returned appropriate response for expired tokens.

Implementation: This functionality is configured in the Program.cs file using JwtBearerEvents.

Project Structure

```

JwtAuthDemo
|
|— Controllers
|   |— AuthController.cs
|   |— SecureController.cs
|   |— AdminController.cs
|
|— Models
|   |— LoginModel.cs
|
|— appsettings.json
|— Program.cs
|— JwtAuthDemo.csproj

```

Result

The JWT Authentication and Authorization mechanism was successfully implemented in the ASP.NET Core Web API application. The application supports secure login, JWT token generation, protected API endpoints, role-based authorization, and token expiry validation. The project was built and executed successfully without any errors.

Conclusion

The experiment successfully demonstrated JWT Authentication and Authorization in ASP.NET Core Web API. Secure user authentication, role-based access control, protected endpoints, and JWT token

validation were implemented successfully. This approach enhances the security of Web APIs by ensuring that only authenticated and authorized users can access protected resources.

Screenshots

1. Build Successful (Build: 1 succeeded, 0 failed)
2. Application Running (Now listening on: <https://localhost:7115>, Application started)

This is a complete lab record and is suitable for submission.

